

Instinct Evolution via Mutation Inherits Paper 1's Tool-Agnosticism

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 14, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work formalizes an inheritance edge between Paper 2 (Li, April 2026) and Paper 1 (Li, April 2026) of the Coordination Knowledge Substrate (CKS) theory series. It does not introduce new axioms. Its sole contribution is to articulate, in operational form, the precise inheritance relationship between Paper 2's instinct evolution mechanism — specifically the mutation sub-mechanism governing LLM version changes — and Paper 1's tool-agnosticism commitment, so that downstream work can adopt or argue against the inheritance claim without ambiguity.

Abstract

Paper 2 of the Coordination Knowledge Substrate (CKS) series names three evolution mechanisms — instinct evolution, DNA evolution, and action-feedback evolution — each operating on a different architectural layer and each inheriting from a distinct Paper 1 commitment. This note formalizes the first of those three inheritance edges: instinct evolution via mutation inherits from Paper 1's tool-agnosticism (Claim 5, §7.1). The inheritance is logical and necessary. Paper 1 positioned the LLM as a replaceable host component — an external execution resource that the substrate uses but that does not govern — and committed to an architecture implementable on any host satisfying three minimal requirements regardless of which specific LLM is used. Paper 2 names LLM version changes as *mutation*: the evolution mechanism where the execution environment of the instinct layer changes due to upstream LLM provider updates. Because Paper 1 committed to LLM replaceability, Paper 2's mutation mechanism is not a new architectural claim about the LLM's status; it is the governance elaboration of what Paper 1's host-layer positioning logically requires. What is genuinely new in Paper 2 is the governance machinery — verification substrates, high-stakes pinning to the reasoning layer, and routing strategies for version transitions — and the explicit naming of an environmental asymmetry: mutation is not initiated by governance (unlike directed selection), but governance manages its effects. This note states the inheritance edge precisely, identifies what is preserved and what is new, introduces the three-evolution-mechanism inheritance triple (C1.17, C1.18, C1.19) and their respective Paper 1 parent commitments, provides an operational test for the host-layer separation the inheritance depends on, and states the prior-art

significance.

1. Why this inheritance edge needs to be stated precisely

Paper 2 introduces three evolution mechanisms as architectural commitments. The mechanisms are not independent inventions; they are extensions of Paper 1 commitments. Stating the inheritance edges precisely matters for two reasons.

First, the defensive-publication purpose of this note series requires that each inheritance edge be on record as public prior art: a claimant who attempts to patent any of Paper 2's evolution governance instruments as novel inventions must contend with the publicly dated chain from Paper 1 through this note to Paper 2. The chain is only legally effective if it is explicit.

Second, the inheritance is not obvious from surface description alone. Paper 1 commits to tool-agnosticism — a host-layer architectural property about which environments can run a CKS substrate. Paper 2 commits to instinct evolution — a named governance mechanism for managing LLM version changes. These appear to operate on different objects (host environments versus evolution dynamics), and without the inheritance analysis the connection can be missed or contested. This note provides that analysis.

The key step in the analysis is the observation that Paper 1's tool-agnosticism positioned the LLM as a *host-layer component*, not a governance actor. The LLM mediates substrate operations; it does not own or author the governance architecture. Because the LLM is host-layer, it is in principle replaceable — one LLM can be exchanged for another without changing the DNA layer, the action layer, the lineage chain, or the orchestration rules. Paper 2's mutation mechanism names the governance of exactly this substitution event: what happens when the host-layer LLM component is updated by its provider. If you commit to LLM replaceability (Paper 1), you must govern LLM version transitions (Paper 2). The inheritance is direct.

2. The inherited foundation: Paper 1's host-layer positioning

Paper 1 (§7.1) commits to tool-agnosticism through three minimal requirements: any environment satisfying persistent structured state, human read/write access, and LLM access to substrate content can host a CKS substrate, with no additional environmental capability required. The third requirement names the LLM as the *mediator* — the external resource that reads substrate content as input to its operations and writes substrate content as output under human governance and human-authored orchestration rules.

Several architectural consequences follow from this positioning, each of which matters for the C1.17 inheritance:

The LLM is host-layer, not governance-layer. In the CKS pattern, governance is exercised over the substrate: humans hold the rights to inspect, modify, and override substrate content and orchestration rules at any time. The LLM operates *under* those orchestration rules, not alongside or above them. The LLM does not hold governance rights; it holds execution permission. This places the LLM on the same architectural tier as the host environment — it is a resource the substrate uses, external to the governance architecture itself.

LLM replaceability is an architectural commitment, not a feature. Because the three requirements characterize the *host interface* rather than a specific host implementation, a CKS substrate that migrates from one LLM to another satisfying Requirement 3 remains architecturally the same object. The governance architecture — DNA layer content, orchestration rules, lineage chains, action layer records — is unchanged. Only the execution behavior may change. The architecture does not depend on LLM identity stability.

No LLM-specific governance instruments appear in Paper 1. Paper 1 names the LLM as a replaceable mediator and commits to the architecture functioning across LLM choices, but it does not commit to specific instruments for managing the transition when an LLM version changes. The host-layer replaceability is the commitment; the governance of the transition is left unspecified. This is precisely where Paper 2 extends.

3. What is preserved: the inherited properties

When Paper 2 names mutation as the governance mechanism for LLM version changes, it inherits three properties directly from Paper 1’s tool-agnosticism commitment.

LLM as replaceable host. Paper 2’s characterization of mutation — LLM upgrades arriving when the LLM provider releases a more capable model, with the system consuming the new model under verification rather than authoring it (§7.2) — presupposes exactly what Paper 1 committed to: that the LLM is a substitutable external component. Mutation is not a threat to the architecture’s coherence; it is an expected event within the architecture because Paper 1 positioned the LLM as something the substrate uses rather than something the substrate is.

Governance persists through LLM changes. Paper 1’s commitment that governance architecture is host-independent means that when a mutation event occurs — when the LLM provider updates the model and the system begins using the new version — the DNA layer, the action layer, the orchestration rules, and the lineage chain are all unchanged. The substrate does not need to be rewritten; the governance authority of humans over substrate content is not disrupted. Paper 2 inherits this property directly and names the layer structure within which it operates.

Architecture-level vs. execution-level separation. Paper 1 committed to a separation between the governance architecture (operating at the substrate level) and the LLM execution

environment (operating at the host level). Paper 2 carries this separation into its evolution framework: mutation crosses from execution level to architecture level only when governance explicitly governs its effects through verification, pinning, or routing decisions made by humans with appropriate authority. Without such governance action, a mutation event changes execution behavior but not architecture.

4. What is new in Paper 2: mutation as named mechanism, governance instruments, and the environmental asymmetry

Paper 1 committed to LLM replaceability. Paper 2 adds four architectural contributions that go beyond what Paper 1 specified, each building on the inherited foundation rather than contradicting it.

Mutation as a named evolution mechanism. Paper 1 acknowledged that the LLM is replaceable; it did not name the governance of LLM version changes as a specific *evolution mechanism* with a distinct architectural role. Paper 2 does. By naming mutation as one of three evolution mechanisms — alongside DNA evolution (directed selection) and action-feedback evolution — Paper 2 commits to mutation having a specific governance shape: undirected, upstream-initiated, managed through responses rather than designed through choice. The naming itself is an architectural commitment: it places LLM version changes in an explicit category that can have its own governance instruments, rather than treating them as incidental deployment details.

Verification substrates as mutation governance instruments. Paper 2 (§8.2) introduces verification substrates as the governance mechanism for determining whether a new LLM version satisfies the cell’s requirements and for managing the transition. Verification substrate patterns may run new and prior versions in parallel, retire superseded reasoning where appropriate, or retain prior reasoning as a fallback or active verification check. None of these instruments appear in Paper 1. They are new governance commitments that Paper 1’s tool-agnosticism makes architecturally possible — the host-layer positioning ensures the governance architecture remains intact during the transition — but does not itself specify.

High-stakes pinning to the reasoning layer. Paper 2 commits (§8.3) to the property that high-stakes decisions can be architecturally pinned to the reasoning layer (the CKS substrate and its orchestration rules) regardless of how capable the instinct layer becomes. This is a new governance property: it ensures that mutation effects — changes in instinct behavior due to LLM version changes — cannot reach certain decisions until those effects have been verified and the pinning explicitly relaxed by human authority. Paper 1 did not anticipate this property because Paper 1 did not model the instinct/reasoning boundary as a dynamic, governable artifact.

The environmental asymmetry: mutation is not governance-initiated. Paper 2 makes explicit a structural asymmetry that is new relative to Paper 1: mutation is *environmental* — not initiated by governance. LLM providers update their models on their own schedules; governance does not control when mutation events occur. Governance responds to them, manages their effects, verifies whether effects satisfy requirements, and routes execution appropriately during transitions — but governance does not trigger the events. This contrasts with DNA evolution, where governance explicitly initiates changes, proposes them, authorizes them, and defines the selection criteria. The asymmetry is new as an architectural commitment because Paper 1 had no evolution model: it described the architecture at a point in time, not the governance of architectural change over time.

This asymmetry has a practical consequence for the governance instruments. Because mutation is not governance-initiated, the governance response must be reactive and verification-based rather than design-and-deploy. The instruments Paper 2 specifies — verification substrates, parallel-run patterns, retention as fallback, routing strategies for version transitions — are all reactive instruments suited to changes that arrive from outside the governance boundary. They are the logical consequence of combining Paper 1’s host-layer positioning (LLM is external) with the new recognition that external components change over time on their providers’ schedules.

5. The three-evolution-mechanism inheritance triple

C1.17 is the first of three notes that together formalize Paper 2’s evolution framework as a set of inheritance edges from Paper 1. The three evolution mechanisms and their Paper 1 parent commitments are:

C1.17 — Instinct evolution (mutation) $\supset$ tool-agnosticism. Mutation governs LLM version changes arriving from upstream. Its parent commitment in Paper 1 is tool-agnosticism: the LLM is host-layer and replaceable, so the architecture must govern the replacement event. This note.

C1.18 — DNA evolution (directed selection) $\supset$ human-governed authority. DNA evolution governs human-initiated changes to the orchestration substrate under explicit goals and authority architectures. Its parent commitment in Paper 1 is the human-governed commitment: the right to modify orchestration rules is the direct authority that DNA evolution exercises at scale. The inheritance is through the authority structure Paper 1 specified, extended to the multi-level, multi-human, multi-scope governance contexts Paper 2 introduces.

C1.19 — Action-feedback evolution $\supset$ substrate-as-source-of-truth. Action-feedback evolution is the mechanism through which accumulated action layer evidence drives DNA layer refinement — the loop closing action back to governance. Its parent commitment in Paper 1 is substrate-as-source-of-truth: the loop can close through substrate only because the substrate

is the authoritative record; action evidence not captured in substrate cannot reliably inform governance. The inheritance is through the authoritative-substrate property that makes the feedback loop architecturally coherent.

The three parent commitments — tool-agnosticism, human-governed, substrate-as-source-of-truth — are three of Paper 1’s six architectural commitments. That Paper 2’s three evolution mechanisms each inherit from a distinct Paper 1 commitment is not coincidence. Each evolution mechanism operates at the architectural layer that its Paper 1 parent commitment most directly specifies: mutation operates at the host layer (where tool-agnosticism lives), directed selection operates at the governance layer (where human-governed lives), and action-feedback operates at the substrate layer (where source-of-truth lives). The inheritance triple is architecturally coherent across all three pairings.

6. Operational test

A concrete test determines whether a given implementation respects the host-layer separation that the C1.17 inheritance depends on. The test probes whether an LLM version change crosses from execution level to architecture level only through explicit governance action.

Test event: The LLM provider releases a new model version. The system begins using the new version.

Pass conditions:

1. The DNA layer content — orchestration rules, harness substrates, governance parameters — is unchanged after the version change, without any migration step required to preserve it.
2. The action layer records — the history of task instances, lineage chains, decisions, and rationale — are unchanged and remain addressable after the version change.
3. Humans with appropriate access can read and modify DNA layer content and action layer records in the same way before and after the version change.
4. Any change to execution behavior observable after the version change is attributable to the new LLM version operating over the same substrate, not to any change in the substrate content itself.
5. High-stakes decisions that were architecturally pinned to the reasoning layer before the version change remain pinned after it, until governance explicitly relaxes the pinning.
6. The governance decision about whether to accept, reject, route around, or roll back the new LLM version is made by humans operating on verification substrate results — not by the LLM itself or by the system automatically.

Fail conditions:

- The DNA layer must be migrated or rewritten to remain compatible with the new LLM version: the substrate depends on LLM-specific properties Paper 1’s architecture committed not to require.
- High-stakes decisions automatically route to the new LLM version without verification: mutation effects reach the architecture without passing through governance.
- The verification decision about the new version is delegated entirely to automated comparison, with no human authority over the outcome: the human-governed property is not preserved through the mutation event.

An implementation that passes all six conditions instantiates the host-layer separation the C1.17 inheritance depends on. An implementation failing any of them may still be useful, but it is not instantiating the CKS architecture’s commitment that LLM version changes are host-layer events managed through governance rather than architectural disruptions.

7. Prior-art significance

This note forecloses three classes of adversarial claims:

Claim class (a): Governing LLM version changes as an evolution mechanism is novel relative to Paper 1’s tool-agnosticism. This note demonstrates that Paper 1’s tool-agnosticism commitment — specifically the positioning of the LLM as a replaceable host-layer component — is the direct architectural foundation for treating LLM version changes as a governance-managed event. The extension from “LLM is replaceable” to “LLM version changes are governed as mutation” is a logical elaboration, not an independent invention.

Claim class (b): Verification substrates as governance instruments for LLM version changes are novel. Verification substrates are new in Paper 2, as §4 of this note acknowledges. But their novelty is bounded: they are the logical governance instruments for upstream-initiated changes arriving at a host-layer component whose replaceability was already committed to in Paper 1. The instrument class — reactive verification over changes the governance architecture does not initiate — is the direct consequence of the inherited host-layer positioning combined with the new evolution model. Any claim that this instrument class was invented independently of the Paper 1 foundation must account for this note’s analysis.

Claim class (c): The concept of environmental evolution — evolution not initiated by governance — is novel. The environmental asymmetry (mutation is not governance-initiated) is genuinely new in Paper 2 as an explicit architectural commitment. However, the asymmetry follows directly from Paper 1’s host-layer positioning: an LLM provider is not part of the governance architecture; updates it makes to its models are therefore, by definition, external to

the governance architecture. The concept of environmental evolution is not a fresh theoretical contribution; it is what Paper 1's externality commitment implies when time and version changes are added to the model.

Source papers

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Instinct Evolution via Mutation Inherits Paper 1's Tool-Agnosticism*. May 14, 2026. ORCID: 0009-0004-8065-3235.